

User Interface(UI) and User Experience(UX)

A few years ago, Amazon (who had then patented its 'one-click-checkout') had a simple buying process. You browsed for items on the website, added them to cart, and then once you clicked on the cart icon, there was a button labelled 'checkout'. Once you click the 'checkout' button, you were presented with a simple form that asked you to login or register. A large number of carts were abandoned at this point, simply because users were not willing to 'register' and were irritated that they had to do so to make a purchase.

Note that Amazon did not ask for additional information on the form, but even then had to suffer such losses. They quickly came up with a 'continue' button and a message saying: "You do not need to create an account to make purchases on our site. Simply click "Continue" to proceed to checkout. To make your future purchases even faster, you can create an account during checkout." This little change resulted in a 45% increase in completed purchases, and for the first year, the site saw an additional \$300M in revenue.

UI/UX, while usually designed as an afterthought, is definitely one of the biggest deal breakers to any application that faces end customers on a regular basis. It is then perhaps of paramount importance to ensure that a smooth and fluid UI coupled with an amazing experience is always present in the application to retain users.

Even exception handling and error pages can be path breaking and wow the customers in a way that keeps them coming for more (Google jump game for no internet connection page).

A beautiful and engaging UI usually has a clean and fluidic nature, with compatibility across all major browsers (yes, even Internet Explorer) and is also capable of restructuring the content across to automatically fit the smaller screen on a tablet or mobile.

Don't implement then architect (a lot)

Once you have your choice of technology stack in place and your dev team is ready with their choice of programming language, development for the application can begin. However, it is possible that at a later stage of iteration you realise that the stack of technologies, while perfect for their use, do not cooperate with each other very well.

For example, a database of choice might handle asynchronous calls well while the UI end technology might assume to make synchronous calls to fetch data to display to the user. In such a case, the application tier is expected to miraculously come up with a "bridge" between these two.